1. INTRODUCTION

1.1 INTRODUCTION TO ALGORITHM VISUALISER

Algorithms and data structures as an essential part of knowledge in a framework of computer science1 have their stable position in computer science curricula2, since every computer scientist and every professional programmer should have the basic knowledge from the area. With the increasing number of students in Central European's higher education systems in last decades (more concrete numbers and impacts for the case of Slovak one can be found in), introduction of appropriate methods into the process of their education is also required. Our scope here is the higher education in the field of computer science.

So within the paper, we discuss the extension of standard methods of teaching algorithms, using the whiteboard or slides, with the algorithm visualizations. According to they can be used to attract students' attention during the lecture, explain concepts in visual terms, encourage a practical learning process, and facilitate better communication between students and instructors. Interactive algorithm visualizations allow students to experiment and available nowadays, and results are quite encouraging. A systematic meta-study of 24 experimental studies can be found in. Results of empirical study aimed at the determination of factors influencing the effectiveness of algorithm visualization are published in. Another example is the study with the objective to determine learning advantage of the interactive prediction facility provided by the courseware containing algorithm animations and data structure visualizations. Based on above mentioned reasons, results of studies carried, as well as our own experiences and explorations, we consider algorithm visualization important and perspective area of further research and application of its results in nowadays computer science education. Except the algorithm visualization, the term software visualization is also often used within the papers published in last years. It usually covers both visualization of algorithms and visualization of data structures, but sometimes also another aspects of software (like its development process) are considered, too. Algorithm visualization, as part of software visualization, could be described as "graphical representation of an algorithm or program that dynamically changes as the algorithm runs". An overview of visualization taxonomies, together with an analysis of factors increasing the effectiveness of software visualization, is summarized in .

Even if the beginnings of algorithm visualization date back into 1940's , the greatest development in the area we could observe within the last 20-30 years. Modern approaches to software visualization were brought in the 1980's by the introduction of system BALSA (Brown & Sedgewick, Brown University, USA). Some of contemporary solutions include systems like TRAKLA23, ANIMAL4,

JAWAA5 or Algorithms In Action6. Concise overview of development in the area of software visualization we provided in, so it is not our intention to analyse this topic within the paper.

In addition to the mathematical and empirical analyses of algorithms, there is yet a third way to study algorithms. It is called algorithm visualization and can be defined as the use of images to convey some useful information about algorithms. That information can be a visual illustration of an algorithm's operation, of its per-formance on different kinds of inputs, or of its execution speed versus that of other algorithms for the same problem. To accomplish this goal, an algorithm visualization uses graphic elements points, line segments, two- or three-dimensional bars, and so on to represent some "interesting events" in the algorithm's operation.

1.2 EXISTING SYSTEM

In traditional way of teaching, we discuss the extension of standard methods of teaching algorithms, using the whiteboard or slides

Despite that arrays constitute a fundamental data structure in introductory programming curriculum, only a small amount of research has been directed to the investigation of students' mental models and programming difficulties with arrays [2, 6, 18, 19]. According to a survey of computing educators, loops and arrays are two of the three programming topics of major difficulty for novice students. Du Boulay reported on students' confusion between an array index and its cell; they also have difficulties to deal with arrays that contain indices as array elements. An unpublished authors' investigation in Greek secondary schools, using a sample of 102 students (K-12), confirmed the same misconceptions and learning difficulties; the majority of the students had faulty or incomplete models of the array concept which resulted in misconceptions and serious difficulties in solving simple algorithmic problems which demand the use of array data structure. In general, there are two main categories of visualization systems in CS education: program visualization and algorithm visualization systems. Program Visualization (PV) systems produce direct representations of programming structures.

1.3 PROBLEM STATEMENT

The application of algorithm visualization to education seeks to help students learn algorithms. The available evidence of its effectiveness is decisively mixed. Although some experiments did register positive learning outcomes, others failed to do so. The increasing body of evidence indicates that creating sophisticated software systems is not going to be enough. In fact, it appears that the level of student involvement with visualization might be more important than specific

features of visualization software. In some experiments, low-tech visualizations prepared by students were more effective than passive exposure to sophisticated software system

1.4 PROPOSED SYSTEM

As was already said that sorting is used for solving a wide range of problems. It may be used for further searching or, for example, as part of different complex tasks. We were talking about sorting. But what actually sorting is? Simply said, sorting is a process of rearranging of items, which are possible to compare, in ascending or descending order. In the text we are meaning only ascending order if not stated in a different way. Ascending order means that items in a sequence are arranged from the smallest to the largest item. On the contrary, descending order means positioning from the largest to the smallest item.

Sorting algorithms are divided into two main types :

1. Algorithms of internal sorting – all the data to sort is stored in the internal memory during the sorting process. It is used when the amount of data to sort is known.
2. Algorithms of external sorting – all the data to sort is stored outside the internal memory (e.g. on a hard disk). These algorithms usually combine sorting in the internal memory, merging of sorted parts and saving them to the external memory. In the text we are talking only about the algorithms of internal sorting.

Here are five main techniques which are usually used with the algorithms which use these techniques and are included in the software :

1. Sorting by Insertion – single items from the sequence are put into the right place of the sorted part. Here belongs Insertion Sort.
2. Sorting by Exchanging – swap elements of each pair that are out of order till no more such pairs exist. Here we have Bubble Sort, Cocktail Sort, Quick Sort.
3. Sorting by Selection – method that uses repeated selection. Selection Sort and Heap Sort use this technique.
4. Sorting by Merging – merging smaller parts with the right order. And Merge Sort uses it. 5. Sorting by Distribution – technique which does not use comparisons to sort. It works relying on the knowledge about the set from where data to sort is taken. Data is distributed to some intermediate structures according to values. Radix Sort, Bucket Sort and Counting Sort belong here.

2. LITERATURE SURVEY

Motivation The motivation behind this project is to study how the operations on data structure are performed. So that students can learn various algorithms through animation. To get a clear knowledge about various data structures and their operations on it. It will makes Data structures learning more interesting. The main goal of this project is to implement a system for various sorting algorithm , prime number , binary search game , - for investigation and visualization the best and worst case for every implemented algorithmic rule. Relevance Of Work Every software engineer should have a good understanding of DSA to develop efficient software. Visualizers have a good history of providing effective understanding to the users. Many algorithm visualizers have been developed over the years.

In 2008, paper “AlCo Lab: Architecture of Algorithm Visualization System” concerns the style of script supported algorithm visualization systems for educational purposes, focusing on the support and the improvement that those systems provide in the process Of teaching of an conceptual subject such as algorithms.

In 2019, paper “Towards Developing an Effective Algorithm Visualization Tool for Online Learning” reports a work-in-progress research project at Athabasca University on developing an effective algorithm visualization tool for online learning.

In 2019, paper “Open Interactive Algorithm Visualization ” presents a work-in-progress project In form developing an open interactive algorithm visualization website.

2021, paper “AlgoAssist: Algorithm Visualizer and Coding Platform for Remote Classroom Learning” focuses on "algorithm visualization", which allows a better understanding of its flow and operation. It supports the combination of the lab into a single application dedicated to preassessment, algorithm explanation, visualization, coding, and post-assessment.

In 2021, paper “Algorithm Visualizer ” aims to simplify and deepen the understanding of algorithms operation. Within the paper we talk about the possibility of improving the standard methods of teaching algorithms, with the algorithm visualizations.

3. SYSTEM ANALYSIS

3.1 SYSTEM REQUIREMENTS

3.1.1 SOFTWARE REQUIREMENTS

The functional requirements or the overall description documents include the product perspective and features, operating system and operating environment, graphics requirements, design constraints and user documentation.

The appropriation of requirements and implementation constraints gives the general overview of the project in regards to what the areas of strength and deficit are and how to tackle them.

- HTML
- CSS
- Javascript

3.1.2 HARDWARE REQUIREMENTS

Minimum hardware requirements are very dependent on the particular software .Applications that need to store large arrays/objects in memory will require more RAM, whereas applications that need to perform numerous calculations or tasks more quickly will require a faster processor.

- Operating system : windows,
- Processor : minimum intel i3
- Ram : minimum 4 GB
- Hard disk : minimum 128 GB

3.2 DOMAIN

3.2.1 THE VIEW

All three types of code are contained in one .html file and can be run solely from this file. One of the advantages of HTML is that it is not necessary to include different types of web languages in a single file.

The View is composed of three objects, called sortArea, bar, and pos. These objects do not belong to any specific object called “View,” but instead operate in the space allocated by the tags in the .html file. You can think of this space as the “main” function, which is the first function called in a program. sortArea is the object that generates the bar graph and runs constantly with a timer to keep the individual bars updated. Therefore, when “Step” is called, it updates the current values of the bars to the new ones based on the steps array (discussed later). Then, the sortArea will show the changes by

redrawing the rectangles with different heights that reflect the new values at the next iteration of the timer. The bars update sixty times a second, so the change invoked by the "Step" button is seen instantly. I based the timer interval on a game tutorial I studied on the W3Schools website when learning JavaScript. The bar object is used to represent each piece of data in the sortArea. It includes the attributes color, value, position, height, and sound. By keeping a separate array called bars for the current bars in the bar graph, it is then easy to change any or all the attributes by iterating over the array as necessary. The In Order, Reverse, and Random buttons do this by quickly iterating over the array to update the bars to a new data configuration. Related to the bar object is the pos object (which is short for position). The canvas area that sortArea updates is an x-y coordinate grid of pixels. To make positioning the bars easier, I created this object to assign a coordinate pair to a position number (1-32). Therefore, if I wanted to change the position of a bar, I would only specify the position number. Then, the bar will look up the exact coordinates from that number and move there. For example, position one consists of the coordinate pair (9, 135), which is the bottom left-hand corner of the bar. However, the rectangle object that each bar is drawn from must be given the top left-hand corner coordinates, so the height must also be considered to reposition the bar correctly.

3.2.2 THE MODEL

The Model is comprised of one object, called sorter. This object contains the sorting algorithm code organized as individual methods. A centralized "start" method takes an integer constant as a parameter that the method recognizes as one of the sorting algorithm methods, and then calls the appropriate one. The five buttons labeled as the four sorting algorithms (Selection Sort, Bubble Sort, Insertion Sort, and Merge Sort, Quick sort) on the user interface directly control this object. When the user has selected a data sorting method and clicked on one of the sort-algorithm buttons, the corresponding sorting algorithm method is called, and a trace is generated as the algorithm sorts the data from the current state of the bar graph. This trace is stored as a two-dimensional integer array, called steps, that is public and accessible to the methods related to the buttons on the web interface. The steps array is a kind of interface through which the user interface communicates with the back-end code. After the back-end code computes the trace, the "Start," "Stop," and "Step" buttons can directly control which part of the array is visualized on the canvas. Usually, this loads in time before the user selects either "Start" or "Step" to view the animation. The "Step" button is the key player in controlling the animation by loading the next step on the canvas. A timer constantly redraws the bars to update their values, and the result is an animated sequence of a single bar moving through the others to a position determined by the algorithm selected. The "Start" button simply calls the "Step" button on a timer.

3.2.3 THE CONTROLLER

The Controller consists of all visible buttons on the webpage. These buttons are coded in HTML and directly call a JavaScript method when pressed. The CSS is used for stylizing the appearance, layout, color, and effects of the buttons, along with the canvas. The methods manipulate the state of the Model and perform the necessary updates to prepare and update the View for animation. The buttons are segregated into four groups. The top, blue-bordered buttons are the sorting algorithm buttons anddirectly. After they are called, the Model builds a steps array that is then available for animation. This animation can be controlled by the bottomright buttons: Start, Stop, and Step. These update the current state of the bar graph by pulling new values from the step array. The View visualizes these updates and the result is the bars changing position based on what algorithm was selected. The Step button is the only button in this group that updates the bars directly. The Start and Stop buttons only control a timer that either calls the Step button at a predetermined speed (every quarter of a second) or stops the timer, respectively. The bottom-left gray-bordered buttons also directly control the bars. They change the initial position of the bars to be in order, reversed, or random. The positions of the bars do not actually change. Instead, each button simply loads an array of values that reflects the ordering desired. The View instantaneously updates this change on the canvas. The last group only consists of one button, which is really a picture. The bottom-left
hand corner of the webpage has a speaker icon that toggles between “mute” and “audio.” This turns on and of the sound animation, so the user can choose whether to run it. By default, the sound animation is muted when the web page loads (for reasons of memory and performance issues).

4.MODULE DESIGN

4.1 ENVIRONMENTAL SETUP

This step is chosen as the first step because is focuses mainly on the front-end where the focus is more towards UI/UX(User Interface/User Experience) rather than the business login, hence this step includes the steps for getting the right dependencies, building user friendly UI and an integration module which easily fits with the business logic to make the product ready. Detailed review and insight of this step will be further discussed.

4.2 DESIGNING THE VISUALIZATION LOGIC

Any product is a good product only if it has good business logic or back end because if we design a beautiful UI but their is no data to display then it is useless.

Hence this step also plays a great role in the final result. Where the main focus is not user driven instead it focuses on developing the visualization function and logic to make the user see what actually is happening in real time by developing a way to implement visualization in real time of different data structures like array, tree, graph etc.

Building user friendly UI and an integration module which easily fits with the business logic to make the product ready.

Detailed review and insight of this step will be further discussed. So first we get the simple website layout ready using simple html tags and the most basic html page with empty values.

Which comes with the some basic JS(javascript) files and some documentation to integrate it in any website. To maintain the abstraction within our project structure it is necessary to store the files in separate directory, hence the below project structure works well considering the maintainability and abstraction.

4.3 SKETCH FILE

Main

|

```
| - README.md
|
| - index.html(main)
|
| - js - |
|        | - utility - |
|                      | - index.js(main)
|                      | - sortingOptions
                                   - |
|                                    | - bubble-sort.js
|                                    | - insertion-sort.js
|                                    | - selection-sort.js
|                                    | - quick-sort.js
|                                    | - merge-sort.js
| | - styles - |
            |  -  style.css(main)
| - image
```

4.4 RESULTS AND ANALYSIS

The best way to go about using the tool is to first select the ordering of the data and then select Which algorithm to visualize. When any one of the algorithm buttons are selected, it will sort the data as it appears on the interface. The ordering takes precedence, asselecting the ordering after the algorithm updates the interface momentarily, while the code has already run the initialization with the

previous data set. After conducting the surveys, this sparked some confusion as the algorithm buttons are listed above the ordering buttons in the interface. One student commented on having difficulty trying to start sorting,thinking that it may be the cause of pressing the buttons inthe wrong order, which in turn did not run the animation. The responses of all the students can be found in Appendix D. Overall, there was not a significant advantage in using my animation tool to help learning about sorting algorithms. By looking at the student responses for question 3, which asked if their understanding of a particular algorithm changed after using the tool, 5 of the 13 students (38%) said yes in some way. The other 7 did not find it very helpful, even though most appreciated the idea of the tool. One student, however, gave a false positive to the tool being helpful (whom I did not include in the 5 that said it was helpful). A drawback to the animation is that it only shows the movements without the comparisons that lead to the movements of the data.This student saw how Selection Sort completes quickly compared to the other algorithms, as there are O(n) swaps that take place, which is beneficial in avoiding unnecessary data moves the computer needs to make. In contrast, the process of comparing the data results in a O(n 2) runtime (the slowest overall). Another student noted this discrepancy in response to question that asked for comments and feedback, noting that Merge Sort is the best of the four sorts. Merge Sort has an average runtime of O(n log2 n), which is the best average runtime out there. One way to resolve this would be to integrate visualizing the comparisons as well as the movements. This way, the bars would change color when an algorithm is comparing data, taking up more time in the animation. Selection Sort and Bubble sort use the most comparisons, so their time to complete would slow down and be more appropriate compared to the other algorithms.

5.SYSTEM DESIGN

5.1 SYSTEM ARCHITECHTURE

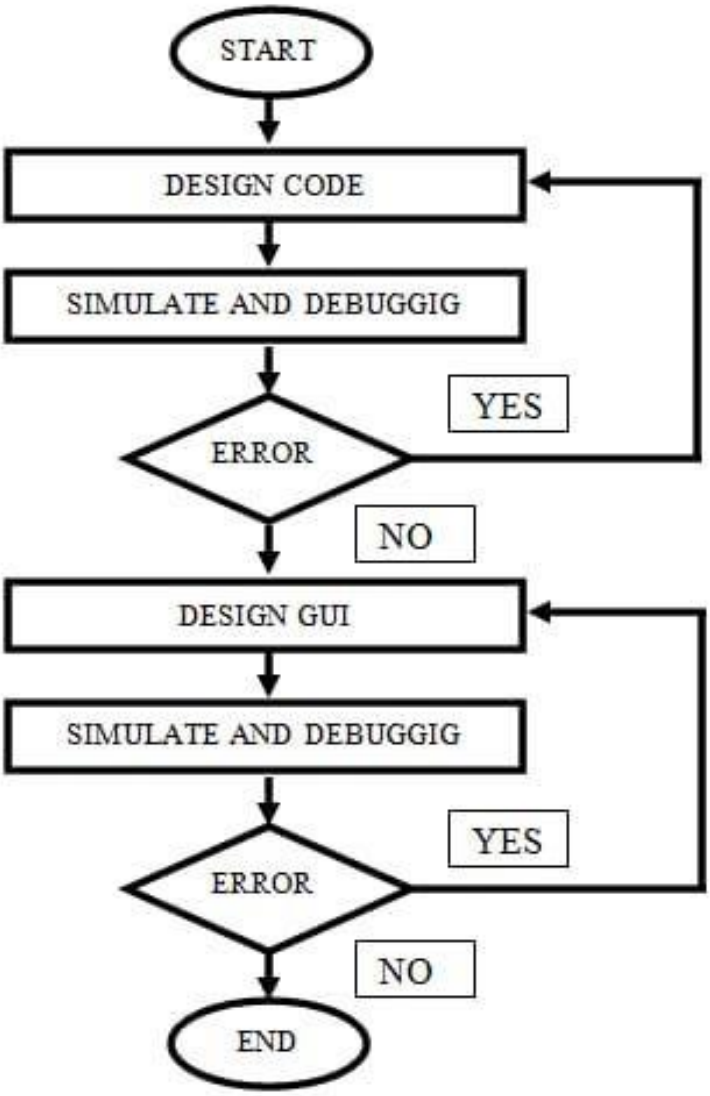

Figure 5.1 Architechture of the system

The back-end code is comprised of HTML5, CSS, and JavaScript. All three types of code are contained in one .html file and can be run solely from this file. One of the advantages of HTML 5 is that it is not necessary to include different types of web languages in a single file. Therefore, each type could have been separated, making a total of three files (plus the miscellaneous sound and image files). This is good practice for readability and keeping related code together. However, I decided not to separate the code for two reasons: 1) to increase the portability of the project by only needing to worry about one project file instead of three, and 2) where in the project file, the change in coding languages is distinctly marked and therefore does not significantly reduce readability. Also, the ability

to put more than one web language in a single file is an example of an RIA (Rich Internet Application).

5.2 UML DIAGRAMS

5.2.1 USE CASE DIAGRAM

In UML, use-case diagrams model the behavior of a system and help to capture the requirements of the system.

Use-case diagrams describe the high-level functions and scope of a system. These diagrams also identify the interactions between the system and its actors. The use cases and actors in use-case diagrams describe what the system does and how the actors use it, but not how the system operates internally.

Use-case diagrams illustrate and define the context and requirements of either an entire system or the important parts of the system. You can model a complex system with a single usecase diagram, or create many use-case diagrams to model the components of the system. You would typically develop use-case diagrams in the early phases of a project and refer to them throughout the development process.

Use-case diagrams are helpful in the following situations:

- Before starting a project, you can create use-case diagrams to model a business so that all participants in the project share an understanding of the workers, customers, and activities of the business.
- While gathering requirements, you can create use-case diagrams to capture the system requirements and to present to others what the system should do.
- During the analysis and design phases, you can use the use cases and actors from your use- case diagrams to identify the classes that the system requires. During the testing phase, you can use use-case diagrams to identify tests for the system.

The following topics describe model elements in use-case diagrams:

- **Usecases**

  A use case describes a function that a system performs to achieve the user's goal. A use case must yield an observable result that is of value to the user of the system.

- **Actors**

  An actor represents a role of a user that interacts with the system that you are modeling. The user can be a human user, an organization, a machine, or another external system.

- **Subsystems**

  In UML models, subsystems are a type of stereotyped component that represent independent, behavioral units in a system. Subsystems are used in class, component, and use-case diagrams to represent large-scale components in the system that you are modeling.

- **Relationships in use-case diagrams**

  In UML, a relationship is a connection between model elements. A UML relationship is a type of model element that adds semantics to a model by defining the structure and behavior between the model elements.

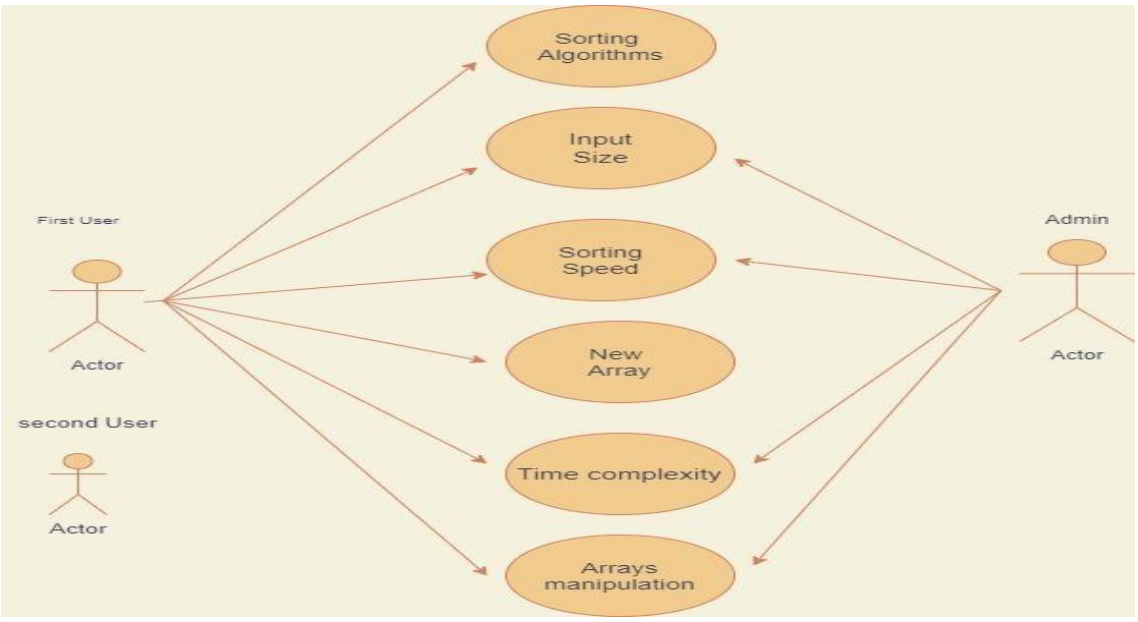

Figure 5.2.1:Use case diagram

5.2.2 CLASS DIAGRAM

Class diagram is a static diagram. It represents the static view of an application. Class diagram is not only used for visualizing, describing, and documenting different aspects of a system but also for constructing executable code of the software application.

Class diagram describes the attributes and operations of a class and also the constraints imposed on the system. The class diagrams are widely used in the modeling of object oriented systems because they are the only UML diagrams, which can be mapped directly with objectoriented languages.

Class diagram shows a collection of classes, interfaces, associations, collaborations, and constraints. It is also known as a structural diagram.

Purpose of Class Diagrams

The purpose of class diagram is to model the static view of an application. Class diagrams are the only diagrams which can be directly mapped with object-oriented languages and thus widely used at the time of construction.

UML diagrams like activity diagram, sequence diagram can only give the sequence flow of the application, however class diagram is a bit different. It is the most popular UML diagram in the coder community.

The purpose of the class diagram can be summarized as −

- Analysis and design of the static view of an application.
- Describe responsibilities of a system.
- Base for component and deployment diagrams.
- Forward and reverse engineering.

How to Draw a Class Diagram?

Class diagrams are the most popular UML diagrams used for construction of software applications. It is very important to learn the drawing procedure of class diagram.

Class diagrams have a lot of properties to consider while drawing but here the diagram will be considered from a top level view.Class diagram is basically a graphical representation of the static view of the system and represents different aspects of the application. A collection of class diagrams represent the whole system.

The following points should be remembered while drawing a class diagram −

- The name of the class diagram should be meaningful to describe the aspect of the system.
- Each element and their relationships should be identified in advance.
- Responsibility (attributes and methods) of each class should be clearly identified
- For each class, minimum number of properties should be specified, as unnecessary properties will make the diagram complicated.
- Use notes whenever required to describe some aspect of the diagram. At the end of the drawing it should be understandable to the developer/coder.

- Finally, before making the final version, the diagram should be drawn on plain paper and reworked as many times as possible to make it correct.

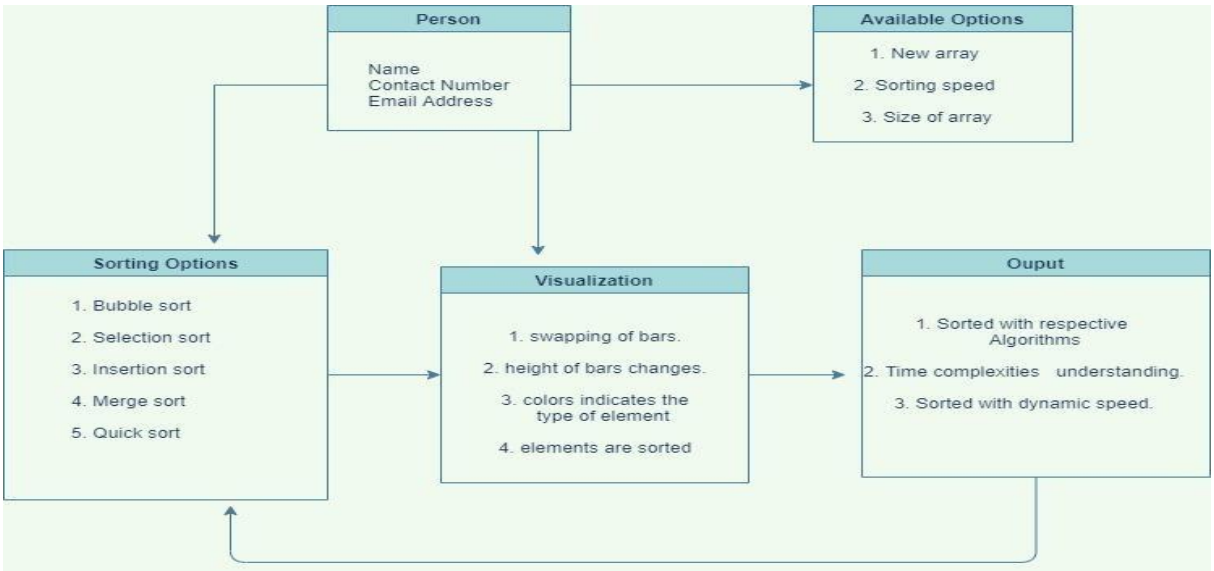

Figure 5.2.2:Class diagram

5.2.3 SEQUENCE DIAGRAM

The sequence diagram represents the flow of messages in the system and is also termed as an event diagram. It helps in envisioning several dynamic scenarios. It portrays the communication between any two lifelines as a time-ordered sequence of events, such that these lifelinessss took part at the run time. In UML, the lifeline is represented by a vertical bar, whereas the message flow is represented by a vertical dotted line that extends across the bottom of the page. It incorporates the iterations as well as branching.

Purpose of a Sequence Diagram

1. To model high-level interaction among active objects within a system.
2. To model interaction among objects inside a collaboration realizing a use case. 3. It either models generic interactions or some certain instances of interaction.

Notations of a Sequence Diagram Lifeline

An individual participant in the sequence diagram is represented by a lifeline. It is positioned at the top of the diagram.

Actor

A role played by an entity that interacts with the subject is called as an actor. It is out of the scope of the system. It represents the role, which involves human users and external hardware or subjects. An actor may or may not represent a physical entity, but it purely depicts the role of an entity.
Several distinct roles can be played by an actor or vice versa.

Activation

It is represented by a thin rectangle on the lifeline. It describes that time period in which an operation is performed by an element, such that the top and the bottom of the rectangle is associated with the initiation and the completion time, each respectively.

Messages

The messages depict the interaction between the objects and are represented by arrows. They are in the sequential order on the lifeline. The core of the sequence diagram is formed by messages and lifelines.

Following are types of messages enlisted below: ○ **Call Message:** It defines a particular communication between the lifelines of an interaction, which represents that the target lifeline has invoked an operation.

- **Return Message:** It defines a particular communication between the lifelines of interaction that represent the flow of information from the receiver of the corresponding caller message.
- **Self Message:** It describes a communication, particularly between the lifelines of an interaction that represents a message of the same lifeline, has been invoked.
- **Recursive Message:** A self message sent for recursive purpose is called a recursive message. In other words, it can be said that the recursive message is a special case of the self message as it represents the recursive calls.
- **Create Message:** It describes a communication, particularly between the lifelines of an interaction describing that the target (lifeline) has been instantiated. ○ **Destroy Message:** It describes a communication, particularly between the lifelines of an interaction that depicts a request to destroy the lifecycle of the target.
- **Duration Message:** It describes a communication particularly between the lifelines of an interaction, which portrays the time passage of the message while modeling a system.

Note

A note is the capability of attaching several remarks to the element. It basically carries useful information for the modelers. **Sequence Fragments**

1. Sequence fragments have been introduced by UML 2.0, which makes it quite easy for the creation and maintenance of an accurate sequence diagram.
2. It is represented by a box called a combined fragment, encloses a part of interaction inside a sequence diagram. 3. The type of fragment is shown by a fragment operator.

Benefits of a Sequence Diagram

1. It explores the real-time application.
2. It depicts the message flow between the different objects.
3. It has easy maintenance.
4. It is easy to generate.
5. Implement both forward and reverse engineering.
6. It can easily update as per the new change in the system.

The drawback of a Sequence Diagram

1. In the case of too many lifelines, the sequence diagram can get more complex.
2. The incorrect result may be produced, if the order of the flow of messages changes.
3. Since each sequence needs distinct notations for its representation, it may make the diagram more complex.

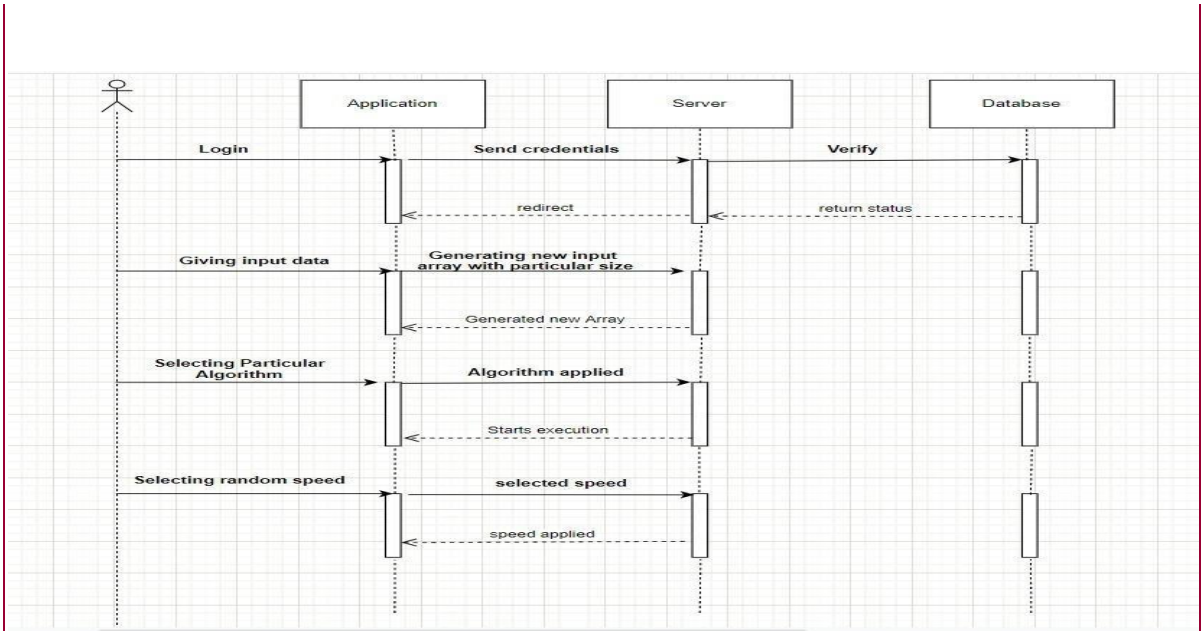

Figure 5.2.3:Sequence diagram

5.2.4 DATA FLOW DIAGRAM

A Data Flow Diagram (DFD) is a traditional visual representation of the information flows within a system. A neat and clear DFD can depict the right amount of the system requirement graphically. It can be manual, automated, or a combination of both.

It shows how data enters and leaves the system, what changes the information, and where data is stored.

The objective of a DFD is to show the scope and boundaries of a system as a whole. It may be used as a communication tool between a system analyst and any person who plays a part in the order that acts as a starting point for redesigning a system. The DFD is also called as a data flow graph or bubble chart.

The following observations about DFDs are essential:

1. All names should be unique. This makes it easier to refer to elements in the DFD.
2. Remember that DFD is not a flow chart. Arrows is a flow chart that represents the order of events; arrows in DFD represents flowing data. A DFD does not involve any order of events.

3. Suppress logical decisions. If we ever have the urge to draw a diamond-shaped box in a DFD, suppress that urge! A diamond-shaped box is used in flow charts to represents decision points with multiple exists paths of which the only one is taken. This implies an ordering of events, which makes no sense in a DFD.
4. Do not become bogged down with details. Defer error conditions and error handling until the end of the analysis.

Standard symbols for DFDs are derived from the electric circuit diagram analysis and are shown in fig:

Circle: A circle (bubble) shows a process that transforms data inputs into data outputs.

Data Flow: A curved line shows the flow of data into or out of a process or data store.

Data Store: A set of parallel lines shows a place for the collection of data items. A data store indicates that the data is stored which can be used at a later stage or by the other processes in a different order. The data store can have an element or group of elements.

Source or Sink: Source or Sink is an external entity and acts as a source of system inputs or sink of system outputs.

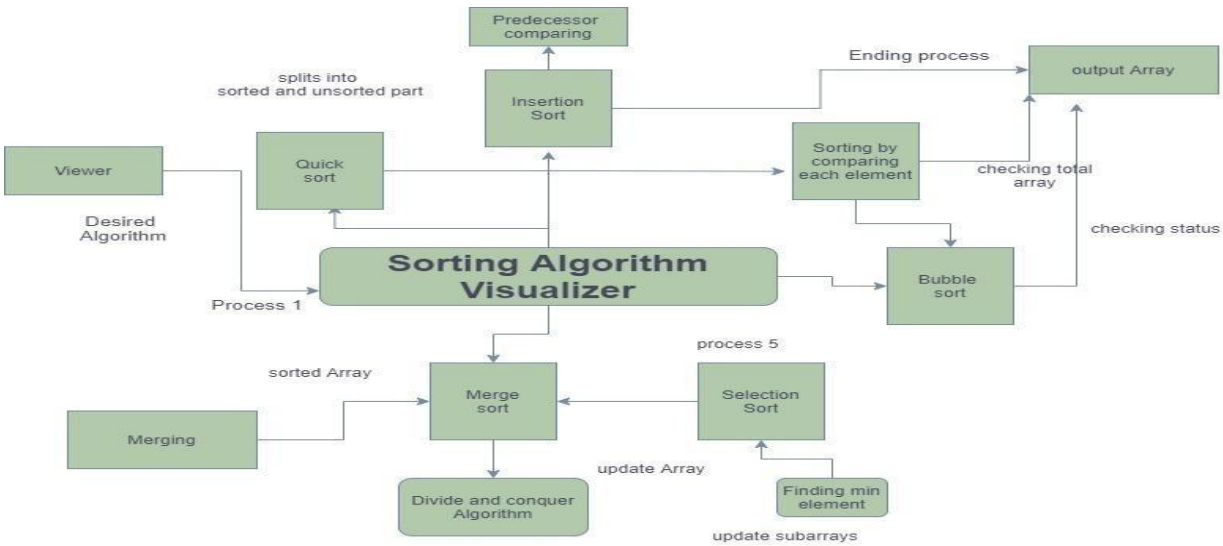

Figure 5.2.4 Data Flow Diagram

5.2.5 ACTIVITY DIAGRAM

Activity diagram is another important diagram in UML to describe the dynamic aspects of the system.

Activity diagram is basically a flowchart to represent the flow from one activity to another activity. The activity can be described as an operation of the system.

The control flow is drawn from one operation to another. This flow can be sequential, branched, or concurrent. Activity diagrams deal with all type of flow control by using different elements such as fork, join, etc

Purpose of Activity Diagrams

The basic purposes of activity diagrams is similar to other four diagrams. It captures the dynamic behavior of the system. Other four diagrams are used to show the message flow from one object to another but activity diagram is used to show message flow from one activity to another.

Activity is a particular operation of the system. Activity diagrams are not only used for visualizing the dynamic nature of a system, but they are also used to construct the executable system by using forward and reverse engineering techniques. The only missing thing in the activity diagram is the message part.

It does not show any message flow from one activity to another. Activity diagram is sometimes considered as the flowchart. Although the diagrams look like a flowchart, they are not. It shows different flows such as parallel, branched, concurrent, and single.

The purpose of an activity diagram can be described as −

- Draw the activity flow of a system.
- Describe the sequence from one activity to another.
- Describe the parallel, branched and concurrent flow of the system.

How to Draw an Activity Diagram?

Activity diagrams are mainly used as a flowchart that consists of activities performed by the system. Activity diagrams are not exactly flowcharts as they have some additional capabilities. These additional capabilities include branching, parallel flow, swim lane, etc.

Before drawing an activity diagram, we must have a clear understanding about the elements used in activity diagram. The main element of an activity diagram is the activity itself. An activity is a

function performed by the system. After identifying the activities, we need to understand how they are associated with constraints and conditions.

Before drawing an activity diagram, we should identify the following elements −

- Activities
- Association
- Conditions
- Constraints

Once the above-mentioned parameters are identified, we need to make a mental layout of the entire flow. This mental layout is then transformed into an activity diagram.

Following is an example of an activity diagram for order management system. In the diagram, four activities are identified which are associated with conditions. One important point should be clearly understood that an activity diagram cannot be exactly matched with the code. The activity diagram is made to understand the flow of activities and is mainly used by the business users

Following diagram is drawn with the four main activities −

- Send order by the customer
- Receipt of the order
- Confirm the order
- Dispatch the order

After receiving the order request, condition checks are performed to check if it is normal or special order. After the type of order is identified, dispatch activity is performed and that is marked as the termination of the process.

Where to Use Activity Diagrams?

The basic usage of activity diagram is similar to other four UML diagrams. The specific usage is to model the control flow from one activity to another. This control flow does not include messages.

Activity diagram is suitable for modeling the activity flow of the system. An application can have multiple systems. Activity diagram also captures these systems and describes the flow from one system to another. This specific usage is not available in other diagrams. These systems can be database, external queues, or any other system.

We will now look into the practical applications of the activity diagram. From the above discussion, it is clear that an activity diagram is drawn from a very high level. So it gives high level view of a system. This high level view is mainly for business users or any other person who is not a technical person.

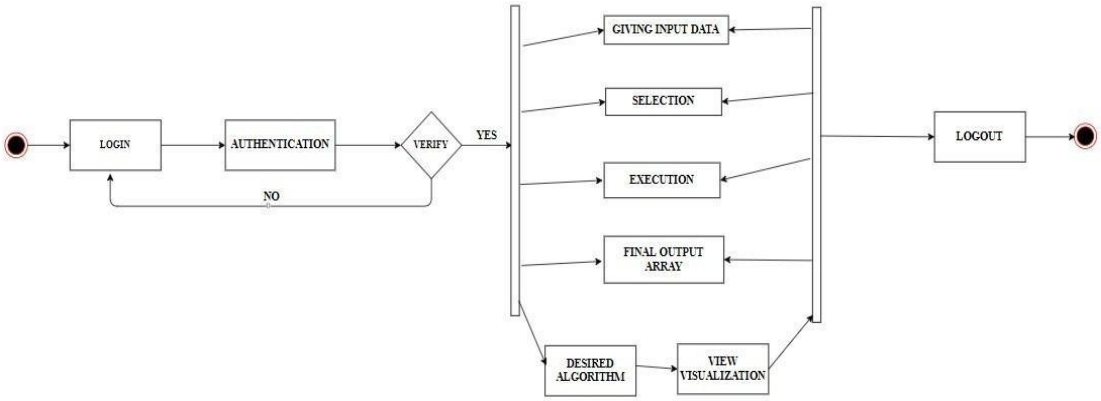

Fig 5.2.5 Activity Diagram

5.2.6 COLLABORATION DIAGRAM

The collaboration diagram is used to show the relationship between the objects in a system. Both the sequence and the collaboration diagrams represent the same information but differently. Instead of showing the flow of messages, it depicts the architecture of the object residing in the system as it is based on object-oriented programming. An object consists of several features. Multiple objects present in the system are connected to each other. The collaboration diagram, which is also known as a communication diagram, is used to portray the object's architecture in the system.

Notations of a Collaboration Diagram

Following are the components of a component diagram that are enlisted below:

1. **Objects:** The representation of an object is done by an object symbol with its name and class underlined, separated by a colon. In the collaboration diagram, objects are utilized in the following ways: ○ The object is represented by specifying their name and class. ○ It is not mandatory for

every class to appear. ○ A class may constitute more than one object. ○ In the collaboration diagram, firstly, the object is created, and then its class is specified.

- ○ To differentiate one object from another object, it is necessary to name them.

2. **Actors:** In the collaboration diagram, the actor plays the main role as it invokes the interaction. Each actor has its respective role and name. In this, one actor initiates the use case.
3. **Links:** The link is an instance of association, which associates the objects and actors. It portrays a relationship between the objects through which the messages are sent. It is represented by a solid line. The link helps an object to connect with or navigate to another object, such that the message flows are attached to links.
4. **Messages:** It is a communication between objects which carries information and includes a sequence number, so that the activity may take place. It is represented by a labeled arrow, which is placed near a link. The messages are sent from the sender to the receiver, and the direction must be navigable in that particular direction. The receiver must understand the message.

When to use a Collaboration Diagram?

The collaborations are used when it is essential to depict the relationship between the object. Both the sequence and collaboration diagrams represent the same information, but the way of portraying it quite different. The collaboration diagrams are best suited for analyzing use cases.

Following are some of the use cases enlisted below for which the collaboration diagram is implemented:

1. To model collaboration among the objects or roles that carry the functionalities of use cases and operations.
2. To model the mechanism inside the architectural design of the system.
3. To capture the interactions that represent the flow of messages between the objects and the roles inside the collaboration.
4. To model different scenarios within the use case or operation, involving a collaboration of several objects and interactions.
5. To support the identification of objects participating in the use case.
6. In the collaboration diagram, each message constitutes a sequence number, such that the toplevel message is marked as one and so on. The messages sent during the same call are denoted with the same decimal prefix, but with different suffixes of 1, 2, etc. as per their occurrence.

Steps for creating a Collaboration Diagram

1. Determine the behavior for which the realization and implementation are specified.
2. Discover the structural elements that are class roles, objects, and subsystems for performing the functionality of collaboration.
   - Choose the context of an interaction: system, subsystem, use case, and operation.
3. Think through alternative situations that may be involved.
   - Implementation of a collaboration diagram at an instance level, if needed. ○ A specification level diagram may be made in the instance level sequence diagram for summarizing alternative situations.

Benefits of a Collaboration Diagram

1. The collaboration diagram is also known as Communication Diagram.
2. It mainly puts emphasis on the structural aspect of an interaction diagram, i.e., how lifelines are connected.
3. The syntax of a collaboration diagram is similar to the sequence diagram; just the difference is that the lifeline does not consist of tails.
4. The messages transmitted over sequencing is represented by numbering each individual message.
5. The collaboration diagram is semantically weak in comparison to the sequence diagram. 6. The special case of a collaboration diagram is the object diagram.

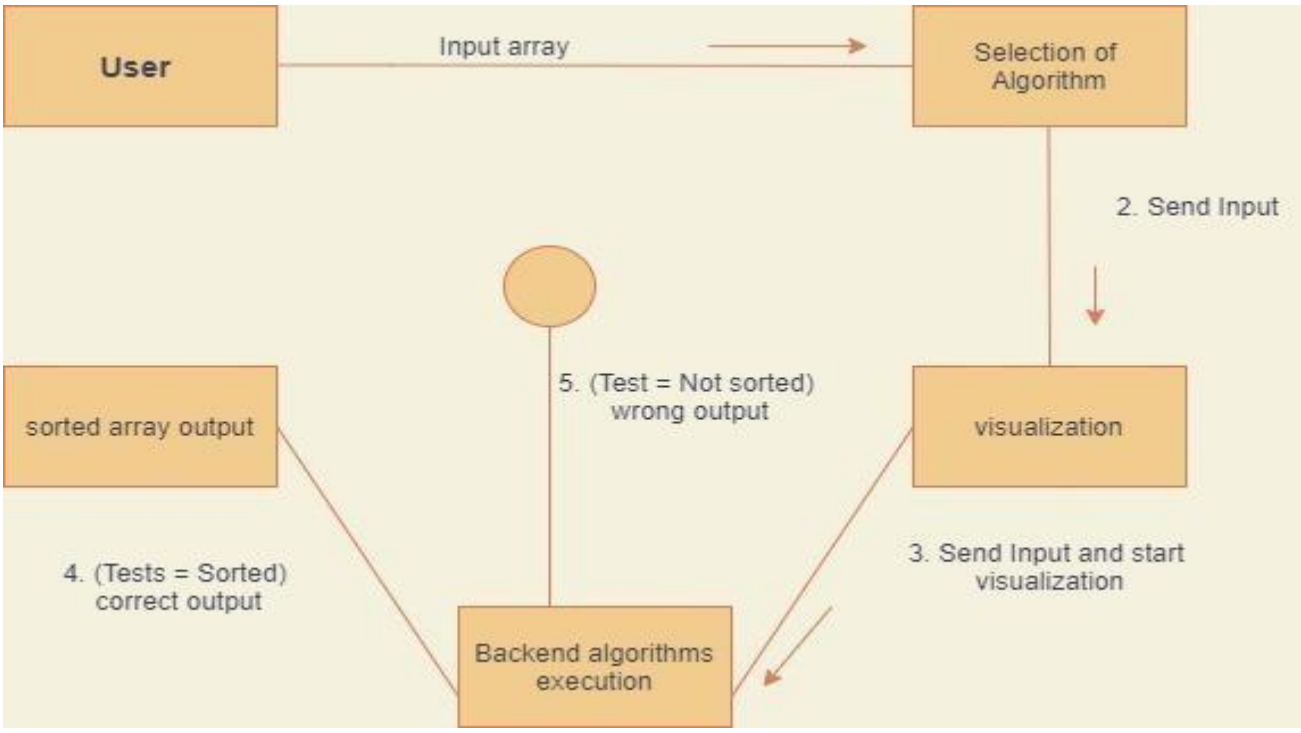

Fig 5.2.6 Collaboration diagram

6. IMPLEMENTATION

Algorithms is the very fundamental about programming which is very important for beginner Developers and getting it cleared is really tough for certain people. So this implementation will focus on getting the right thing to large number of people in easy and efficient manner. And here comes the HTML. Website is the easiest and best way of getting to the most of the audience there is always an option of desktop app or mobile application,but those will limit the reach-ability of the idea hence the idea will be implemented using website.Where we will be having website which will be user friendly demonstrating most of the generic algorithms where user can interact and see the algorith happening infant of them rather than visualize it in dry run or imagining and getting the theory clear**.**

This implementation is broadly divided into 2 steps which is decided based on various factors, considering the facts that all these three steps can be implemented parallel without disturbing the progress of other. Making the development process smooth and fast reaching our end goal is also the important factor which can't be ignored.

SOURCE CODE :

```
<!DOCTYPE html>
<html lang="en">
<head>

        <meta charset="UTF-8">
        <meta http-equiv="X-UA-Compatible" content="IE=edge">
        <meta name="viewport" content="width=device-width, initial-scale=1.0">
        <title>Sorting Visualizer</title>
        <link rel="shortcut icon" href="./styles/images/icon.png" type="image/x-icon">
        <link rel="stylesheet" href="./styles/css/bootstrap.min.css">
        <link rel="stylesheet" href="./styles/style.css">

</head>
<body>

        <!-- container for sorting options -->
        <div class="sortingoptions">
                <!-- sub container for positioning -->
```

```
                <div class="button-container">
                        <!-- buttons -->
  <button type="button" class="btn btn-outline-primary btns bouncy bubblesort">Bubble Sort</button>
  <button type="button" class="btn btn-outline-primary btns bouncy insertionsort">Insertion Sort</button>
  <button type="button" class="btn btn-outline-primary btns bouncy selectionsort">Selection Sort</button>
  <button type="button" class="btn btn-outline-primary btns bouncy quicksort">Quick Sort</button>
  <button type="button" class="btn btn-outline-primary btns bouncy mergesort">Merge Sort</button>
                </div>
        </div>

        <!-- container for the manipulation section -->
        <div class="manipulation">

                <!-- buttons and inputs -->

 <button type="button" class="btn btn-outline-primary btns createArray">New Array</button>

 <button type="button" class="btn btn-outline-primary btns arraySizeBtn">Array Size</button>
                <input class='arraySize' type="range" value=30 min=2 max=100></input>

 <button type="button" class="btn btn-outline-primary btns sortingSpeedBtn">Sorting Speed</button>
                <input class='sortingSpeed' type="range" value=14 min=2 max=50></input>

        </div>


        <!-- container for bar field -->
        <div class="bars"></div>
```

```
        <!-- container for description -->
        <div class="algos">
                <!-- description comes here -->
        </div>

        <!-- source tags -->
        <script src="./js/bootstrap/bootstrap.min.js"></script>
        <script src="./js/utility/index.js"></script>
        <script src="./js/utility/sortingOptions/bubble-sort.js"></script>
        <script src="./js/utility/sortingOptions/insertion-sort.js"></script>
        <script src="./js/utility/sortingOptions/selection-sort.js"></script>
        <script src="./js/utility/sortingOptions/quick-sort.js"></script>
        <script src="./js/utility/sortingOptions/merge-sort.js"></script>

</body>
</html>
```

7. OUTPUT SCREENS

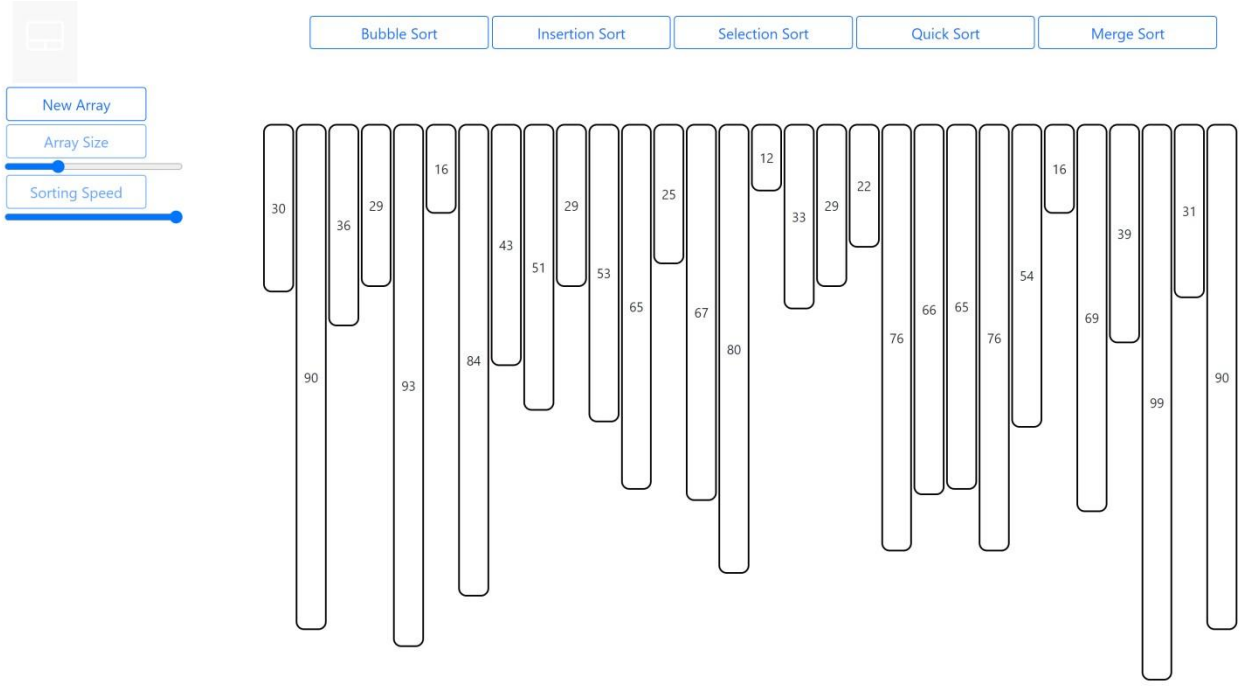

Fig 7.1:NORMAL APPEARANCE

Fig 7.2:BUBBLE SORT

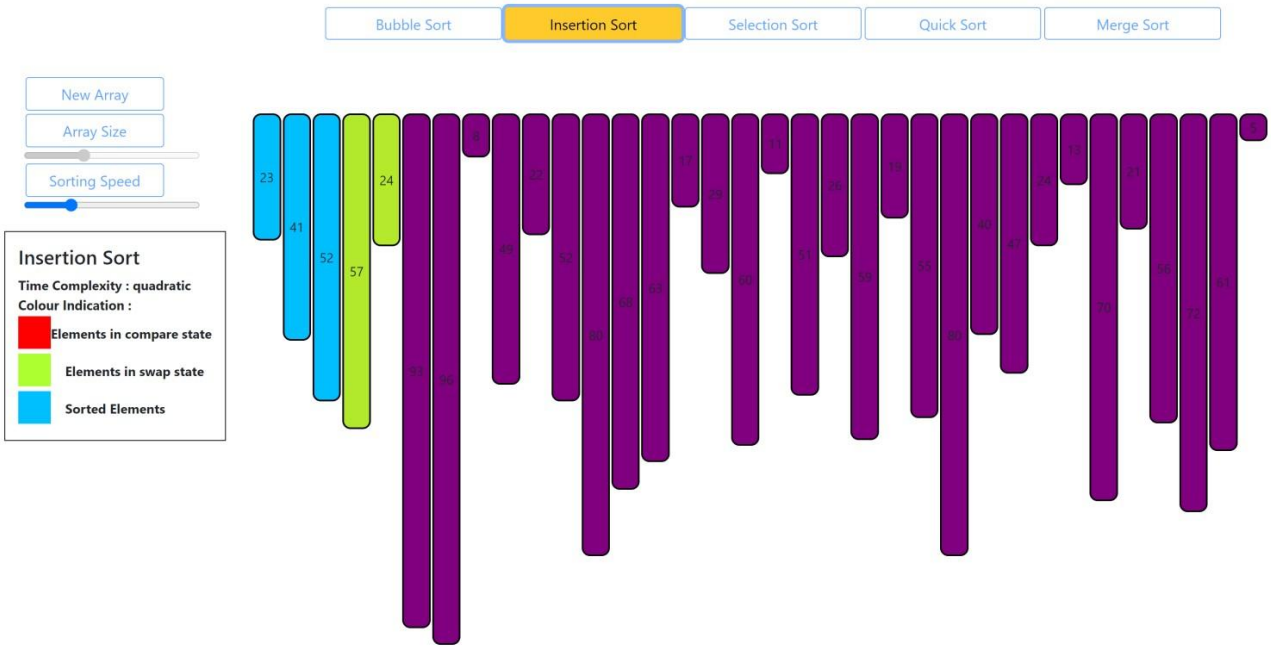

Fig 7.3:INSERTION SORT

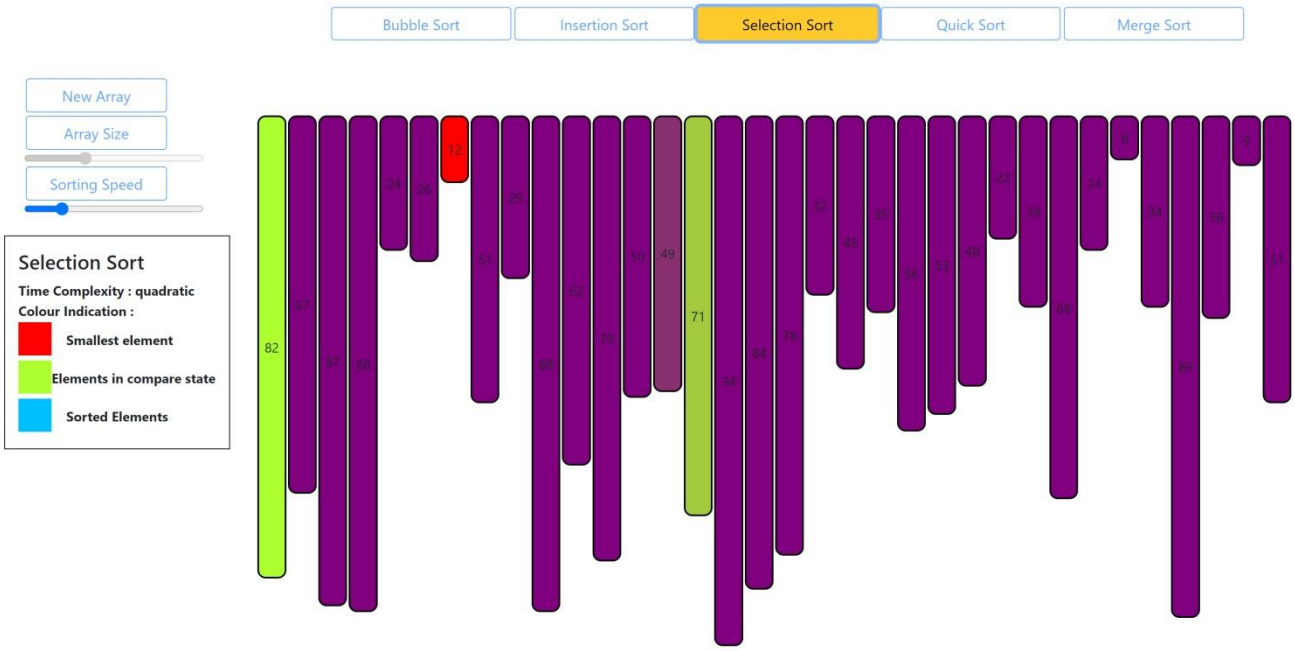

Fig 7.4: SELECTION SORT

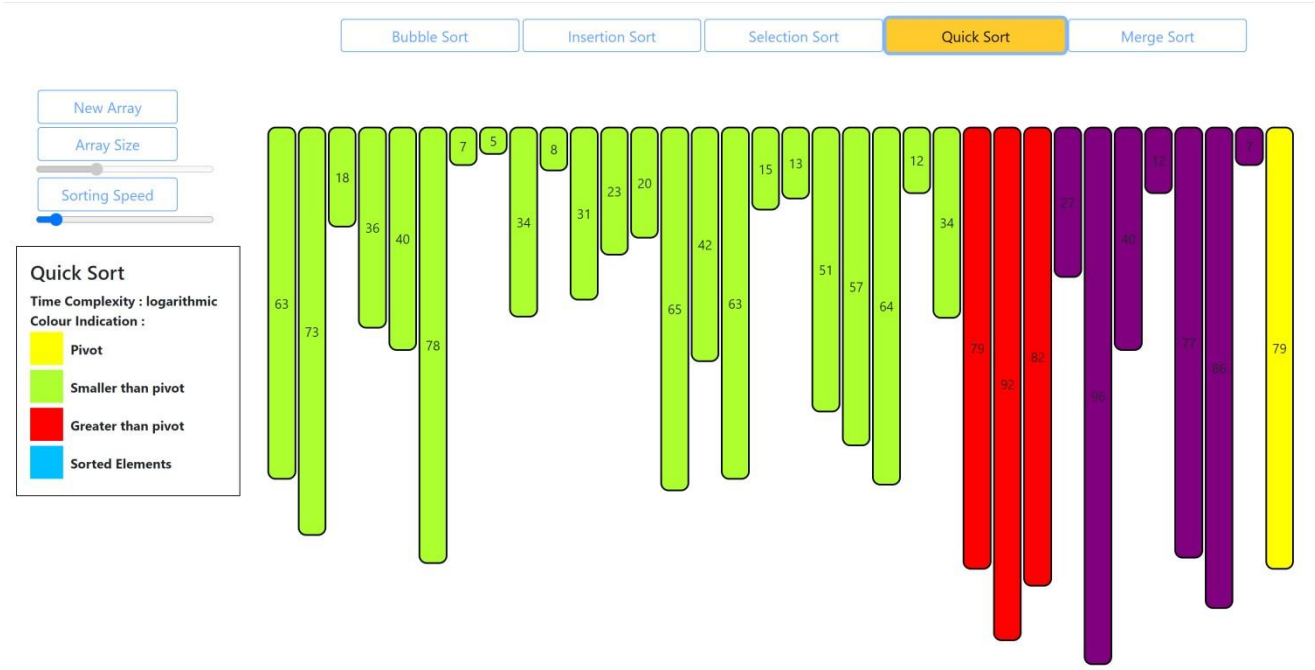

Fig 7.5: QUICK SORT

Fig 7.6: MERGE SORT

8. TESTING

In order to perform an experiment to evaluate my animation tool, I recruited coworkers to act as test subjects and survey respondents. To that end, the plan was to expose her Computer Science Data Structures class to the tool and get them to take part in a survey comprising questions that gave the students the ability to write down what they observed and learnt, even if it was little. The questions contained in Appendix C are the questions which were utilized for the survey. Despite the class size being of twenty-one students, only thirteen were present; fortunately, all of the students who had completed the survey were there. The set-up here was ideal since it addressed the audience for whom I was designing this course: a group of college students who were taking their first course in computer science and who needed assistance with algorithms. a) Results Start by arranging the data, and then pick the visualization algorithm to use. Algorithm buttons provide sorting of data as it arrives on the interface. Asking to specify the ordering of elements takes precedence because when the algorithm has completed running the initialization process, the interface is now showing a new ordering, while the code has already completed running the initialization with the prior data set. There was considerable confusion caused by the way the ordering buttons and algorithm buttons were shown in the UI after the surveys were completed. When beginning the sorting process, the student noted that she was having problems starting because she believed that she was hitting the buttons in the wrong order. This then led to her failing to execute the animation. The answers can be found in Appendix D. Overall, my animation tool did not aid with the understanding of sorting algorithms. Among those who answered question 3, which questioned if their knowledge of a particular algorithm changed after using the tool, 5 of the 13 students (38%) stated that they had in some way altered their previous knowledge of the algorithm. Many thought the tool was a good concept, while the other 7 did not find it useful at all. It was said that one student stated a false positive about the instrument (whom I did not include in the 5 that said it was helpful).

8.1 DESIGNING THE EXPERIMENT

As this tool is designed to be a learning aid in the classroom, this has presented ethical issues in experiment design. I briefly mentioned this earlier in Chapter 3 as being a part of the paper "Do Algorithm Animations Assist Learning?: An Empirical Study and Analysis" [6]. The students in Stasko's study were randomly separated into two groups. In one, the students were taught the material using conventional methods (i.e. textbook). In the other, the students had both the textbook and an animation tool to assist them when learning the material. This separation is necessary for more reliable results that will hopefully help improve teaching and learning for manstudents, but this comes at the cost of giving the participants in the study different experiences. Although Stasko's results proved a non-significant difference in the post-tests of both groups, it still created a noticeable learning gap, giving the students that used the animation tool an advantage over the others.

.

9. CONCLUSION

According to our findings, algorithm visualization can be seen as a valuable supporting tool, used in addition to standard ways of education in the field of computer science. Within the paper we provided an overview of the VizAlgo algorithm visualization platform as well as our practical experiences with the system. We believe (and the results of questionnaire support our belief) it helps to improve the quality of education in the field and contribute to the solution for some of the problems in higher education mentioned at the beginning of the paper. There are still open issues with using algorithm visualizations. Algorithm visualizations can help understanding the principles, but do not replace the need to implement algorithms by students is a chosen programming language. Another drawback of using algorithm visualizations within our subject is the lack of the tool offering required visualizations in a single package with the unified interface. The VizAlgo platform can also be considered as a step in this direction. Generally, more systematic evaluation of algorithm visualization tools is required, as there is rather informal evidence available that applications of algorithm visualizations are useful . We summarized results of the questionnaire filled in by students in order to support our decisions on further development of the platform, too. Our intentions here include development of new plugin modules from the area of sorting algorithms and more complex data structures. Some of proposed core-related features are on the list too (like graphically better visualizations, optional changing of algorithm properties), but some of them will probably not be implemented in a near future (like undo/step back in running visualization), as they would require more fundamental changes. Except the extensions mentioned within the questionnaire, we also consider some other interesting features: dynamic changes in algorithm pseudocode reflected in visualization, different visual views on running algorithm or simultaneous comparison of different algorithm visualizations

10. FUTURE SCOPE

With the shift of remote and digital literacy, a combined platform serving the effective literacy requirements of students is required. Algorithm Visualizer is a combined platform that's a comprehensive result for educators and students to educate and learn online effectively. It substantially focuses on “algorithm visualization", which allows a better understanding of its inflow and operation.

11. REFERENCES

[1] K. Mehlhorn, P. Sanders, Algorithms and Data Structures (Springer-Verlag, Berlin Heidelberg, 2008)

[2] J. Genci, Possibilities to Solve Some of the Slovak Higher Education Problems Using Information Technologies, In proceedings of: 10th IEEE International Conference on Emerging eLearning Technologies and Applications, ICETA 2012, Stará Lesná, The High Tatras, Slovakia, November 8-9, 2012

[3] C.D. Hundhausen, S.A. Douglas, J.T. Stasko, A Meta- Study of Algorithm Visualization Effectiveness, J. Visual Lang. Comput. 13, 259–290, 2002 [4] K. Mehlhorn, P. Sanders, Algorithms and Data Structures (Springer-Verlag, Berlin Heidelberg, 2008)

[4] J. Genci, Possibilities to Solve Some of the Slovak Higher Education Problems Using Information Technologies, In proceedings of: 10th IEEE International Conference on Emerging eLearning Technologies and Applications, ICETA 2012, Stará Lesná, The High Tatras, Slovakia, November 8-9, 2012

[5] C.D. Hundhausen, S.A. Douglas, J.T. Stasko, A MetaStudy of Algorithm Visualization Effectiveness, J. Visual Lang. Comput. 13, 259–290, 2002.